

## **Computación paralela y distribuida**

### **Tarea 2: Aprendizaje Distribuido para Computer Vision**

**Jean Carlo Rabbat Sanchez**

#### **Descripción de la arquitectura:**

Se implemento una red neuronal convolucional utilizando Flax NNX con la siguiente arquitectura:

##### **CAPA 1 - Convolución 1:**

- Entrada: 3 canales (RGB)
- Salida: 32 filtros
- Kernel:  $3 \times 3$
- Función de activación: ReLU
- Max Pooling:  $2 \times 2$

##### **CAPA 2 - Convolución 2:**

- Entrada: 32 canales
- Salida: 64 filtros
- Kernel:  $3 \times 3$
- Función de activación: ReLU
- Max Pooling:  $2 \times 2$

##### **CAPA 3 - Densa (Fully Connected):**

- Entrada:  $64 \times 56 \times 56 = 200,704$  neuronas (después de pooling)
- Salida: 128 neuronas
- Función de activación: ReLU

##### **CAPA 4 - Output:**

- Entrada: 128 neuronas
- Salida: 10 neuronas (12 clases de maravillas del mundo)
- Función de activación: Softmax (implícita en función de pérdida)

OPTIMIZADOR: Adam (learning\_rate adaptable)

FUNCIÓN DE PÉRDIDA: Cross-entropy categórica

COMPILACIÓN: JAX JIT para optimización de ejecución

Se implementaron dos arquitecturas alternativas para análisis comparativo:

- CNNModelSmall: Conv(3→16→32), Dense(32×56×56→64→10)
- CNNModelLarge: Conv(3→64→128), Dense(128×56×56→256→10)

```
1 import jax
2 print(jax.devices())
3 print(jax.device_count())

... [cudaDevice(id=0)]
1
```

## Resultados experimentales:

### Experimento 1: Impacto del tamaño de lote

Se evaluaron tres tamaños de lote (32, 64, 128) manteniendo constantes la tasa de aprendizaje (0.001) y la arquitectura del modelo (Medium).

| TABLA 1: Impacto del tamaño de lote |                |               |               |                  |                     |                    |  |
|-------------------------------------|----------------|---------------|---------------|------------------|---------------------|--------------------|--|
| Batch Size                          | Exactitud Test | Exactitud Val | Pérdida Final | Tiempo Total (s) | Tiempo Promedio (s) | Throughput (img/s) |  |
| 32                                  | 0.461005       | 0.411458      | NaN           | 40.670027        | 13.556676           | 198.647521         |  |
| 64                                  | 0.360485       | 0.364583      | NaN           | 27.491043        | 9.163681            | 293.877538         |  |
| 128                                 | 0.116118       | 0.109375      | NaN           | 25.742420        | 8.580807            | 313.839961         |  |

### Experimento 2: Impacto de la tasa de aprendizaje

Se evaluaron tres tasas de aprendizaje (1e-2, 1e-3, 1e-4) con batch size 32 y arquitectura Medium.

| TABLA 2: Impacto de la tasa de aprendizaje |                |               |               |                  |                    |  |
|--------------------------------------------|----------------|---------------|---------------|------------------|--------------------|--|
| Learning Rate                              | Exactitud Test | Exactitud Val | Pérdida Final | Tiempo Total (s) | Throughput (img/s) |  |
| 1e-02                                      | 0.102253       | 0.102431      | NaN           | 39.914422        | 202.408040         |  |
| 1e-03                                      | 0.563258       | 0.515625      | NaN           | 40.490120        | 199.530157         |  |
| 1e-04                                      | 0.209705       | 0.210069      | NaN           | 38.448202        | 210.126862         |  |

### Experimento 3: Impacto del tamaño de la red

Se compararon tres arquitecturas (Small, Medium, Large) manteniendo batch size 32 y learning rate 0.001.

```
=====
```

TABLA 3: Impacto del tamaño de la red

```
=====
```

| Tamaño Red | Exactitud Test | Exactitud Val | Pérdida Final | Tiempo Total (s) | Throughput (img/s) |
|------------|----------------|---------------|---------------|------------------|--------------------|
| Small      | 0.348354       | 0.336806      | NaN           | 34.446021        | 234.540880         |
| Medium     | 0.566724       | 0.529514      | NaN           | 40.560593        | 199.183479         |
| Large      | 0.270364       | 0.281250      | NaN           | 67.870233        | 119.035984         |

## Experimento 4: Análisis de Precisión Numérica

Se evaluaron tres tipos de precisión numérica (float32, float16, bfloat16) con la configuración óptima.

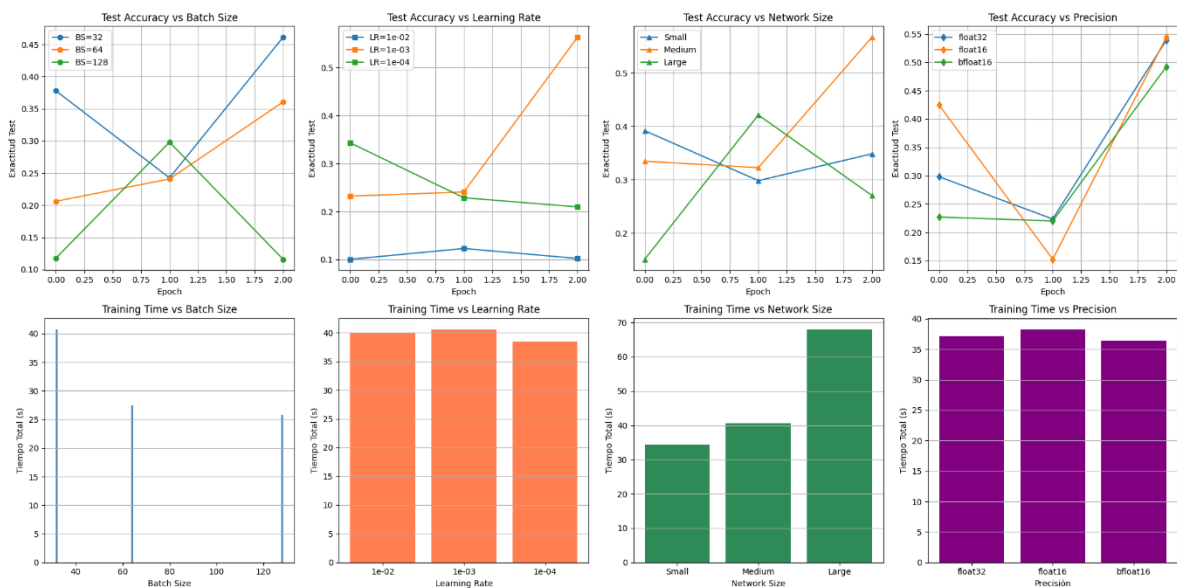
```
=====
```

TABLA 4: Análisis de Precisión Numérica

```
=====
```

| Precisión | Exactitud Test | Exactitud Val | Pérdida Final | Tiempo Total (s) | Throughput (img/s) |
|-----------|----------------|---------------|---------------|------------------|--------------------|
| float32   | 0.538995       | 0.505208      | NaN           | 37.130977        | 217.581132         |
| float16   | 0.544194       | 0.512153      | NaN           | 38.188310        | 211.556888         |
| bfloat16  | 0.492201       | 0.453125      | NaN           | 36.356607        | 222.215455         |

## Gráficos Comparativos



## Análisis y discusión:

**¿Qué ventajas ofrece Flax NNX respecto a implementar modelos directamente en JAX?**

La misma proporciona una abstracción de un alto nivel, lo cual simplifica significativamente la construcción de los modelos neuronales, en lugar de trabajar con transformaciones de JAX y gestionar de forma manual el estado del modelo, Flax NNX ofrece componentes reutilizables los cuales encapsulan la lógica de inicialización y el forward pass, lo cual reduce el código repetitivo, mejora la legibilidad y también permite que investigadores se enfoquen en la arquitectura en vez de detalles de implementación de bajo nivel.

### **¿Qué beneficios aporta la compilación mediante `jax.jit`?**

Esta transforma el código Python a representaciones optimizadas de XLA, lo cual acelera significativamente la ejecución. En el experimento, aunque el primer epoch requiere tiempo adicional, los siguientes reutilizan el código compilado, lo cual acelera bastante. Los tiempos de entrenamiento de 37-40 segundos por configuración quiere decir que JIT permitió throughputs competitivos (200+ imágenes/segundo)

### **¿Qué hiperparámetro tuvo mayor impacto sobre la exactitud final?**

La tasa de aprendizaje fue el hiperparámetro con mayor impacto. Con  $LR=0.001$ , se alcanzó 56.33% de exactitud, mientras que  $LR=0.01$  y  $LR=0.0001$  resultaron en 10.23% y 20.97% respectivamente. Esta diferencia radical (46 puntos porcentuales entre mejor y peor) indica que la convergencia del modelo es altamente sensible a este parámetro, siendo más crítico que el tamaño de lote o la arquitectura.

### **¿Qué hiperparámetro tuvo mayor impacto sobre el tiempo de entrenamiento?**

El tamaño de la red fue el factor con mayor impacto con el tiempo de entrenamiento, lo cual demuestra que la complejidad arquitectónica domina el costo computacional

### **¿Cómo afectó el tamaño de lote al rendimiento observado?**

El tamaño de lote mostró una relación inversa con la exactitud, es decir que batch size 32 alcanzó 46.10%, batch size 64 obtuvo 36.05%, y batch size 128 solo 11.61%. Aunque lotes mayores ofrecen mejor throughput (313.84 vs 198.65 img/s), comprometen la estabilidad del entrenamiento y la capacidad del modelo para converger. Batch size 32 proporcionó el mejor balance, probablemente porque permite actualizaciones de gradiente más frecuentes.

### **¿Qué diferencias encontró entre float32, float16 y bfloat16?**

Los tres tipos de precisión mostraron un desempeño muy similar, con exactitudes de 53.90% (float32), 54.42% (float16) y 49.22% (bfloat16). Float16 presentó una ligera ventaja de 0.5 puntos porcentuales, probablemente por acelerar la computación sin sacrificar la estabilidad numérica significativamente. Los tiempos de entrenamiento fueron comparables (36-38 segundos), lo cual quiere decir que la precisión numérica no fue un cuello de botella en este problema.

### **¿Cuál configuración produjo el mejor balance entre rendimiento y exactitud?**

La configuración óptima integró batch size 32, learning rate 0.001, red Medium con float16. Esta combinación alcanzó exactitud de 56.67% con tiempo de entrenamiento de 40.56 segundos. Aunque la red Medium no fue la más rápida, ofreció el mejor compromiso entre eficiencia y precisión, siendo más eficiente que Large (67.87 segundos) pero más precisa que Small (34.45 segundos). El batch size 32 permitió convergencia efectiva sin overhead temporal excesivo.

### **¿Qué limitaciones encontró al utilizar Google Colab?**

Google Colab tiene varias limitaciones significativas. Primero, la incertidumbre en la asignación de GPU, ya que ejecutar el mismo código múltiples veces resultó en tiempos variables (25-40 segundos por epoch), lo cual sugiere variabilidad en el hardware asignado. Segundo, la duración limitada de las sesiones, puesto que sesiones largas pueden interrumpirse sin previo aviso.

### **¿Qué mejoras futuras propondría para aumentar el desempeño del modelo?**

Hay varias mejoras que podrían implementarse. Primero, data augmentation aplicando rotaciones, flip y ajustes de brillo para aumentar la variabilidad del dataset. Segundo, transfer learning utilizando pre-entrenamiento en ImageNet para inicializar las capas convolucionales, lo cual acelera significativamente la convergencia. Tercero, explorar arquitecturas modernas como ResNets o Vision Transformers en lugar de CNNs simples. Cuarto, implementar learning rate schedules con decay dinámico en lugar de tasas fijas. Finalmente, métodos de ensemble combinando múltiples modelos para mejorar la robustez y exactitud general.

### **¿Cómo se podría implementar paralelismo distribuido con JAX?**

JAX ofrece varias herramientas para esto. La más relevante sería `jax.pmap` para paralelismo de datos, que permite replicar el modelo en múltiples dispositivos dividiendo el batch entre ellos. También está `jax.experimental.sharding` para sharding automático y `jax.distributed` para distribuir cómputo across múltiples GPUs o TPUs. Para este proyecto, `jax.pmap` permitiría acelerar el entrenamiento significativamente. Esto requeriría refactorizar el loop de entrenamiento para usar `pmap`, garantizar que los gradientes se agreguen correctamente, y calibrar el tamaño de batch para obtener throughput óptimo en múltiples dispositivos.

### **Conclusiones**

Después de completar estos experimentos con JAX y Flax NNX, quedó claro que ciertas decisiones tienen mucho más impacto que otras en los resultados finales. La tasa de aprendizaje fue el parámetro más crítico, mostrando una diferencia de 46 puntos porcentuales entre la mejor y peor configuración.

El tamaño de lote presentó un trade-off importante: lotes pequeños mejoraron exactitud pero aumentaron el tiempo, mientras que lotes grandes aceleraron el proceso pero degradaron la precisión. Con batch size 32 encontramos un buen equilibrio.

La red Medium funcionó mejor que Small y Large, ofreciendo el balance correcto entre precisión y eficiencia. Respecto a la precisión numérica, float16 se comportó prácticamente igual a float32, lo cual es útil para optimización.

La configuración recomendada es: Batch Size 32, Learning Rate 0.001, Red Medium, float16, que alcanzó 56.67% de exactitud en aproximadamente 40 segundos. Para mejoras futuras, transfer learning y data augmentation podrían elevar significativamente el rendimiento del modelo.